Reinforcement Learning-Based Optimal Setpoint Determination in Energy Utility Systems

MAIN MESSAGE

A final thesis defense comparing learned setpoints with the optimization baseline.

WHAT TO PUT ON THE SLIDE

- Use the full thesis title, followed by 'Master Thesis - Final Presentation'.
- Add Ashish Thapa; MSc Artificial Intelligence; Brandenburg University of Technology Cottbus-Senftenberg.
- List reviewers Prof. Dr. Ralf Wunderlich and Dr.-Ing. Loukas Kyriakidis; insert the actual defense date.

SUGGESTED VISUAL / LAYOUT

Use a clean title slide with small battery and steam-system symbols. Keep administrative details smaller than the title. The first 30 guide pages cover the core presentation; optional theory follows.

WHAT TO EXPLAIN ALOUD

Introduce the problem in one sentence: selecting economical operating decisions while respecting physical limits. State that the work studies a simpler battery and a more complex industrial steam utility. Allow roughly 25-30 minutes for the planned deck, adjusting to the defense slot.

SOURCE / paper/main.tex: title metadata and Abstract

Presentation roadmap

MAIN MESSAGE

Follow the problem, method, two case studies, and the final engineering conclusion.

WHAT TO PUT ON THE SLIDE

- Motivation, research question, and contributions: slides 3-5.
- Shared method and experimental setup: slides 6-11.
- Battery case: slides 12-18; cookie-factory case: slides 19-27.
- Synthesis, limitations, future work, and questions: slides 28-30.

SUGGESTED VISUAL / LAYOUT

Draw a four-stage horizontal roadmap. Highlight the two case studies in different colors and reuse those colors throughout the deck.

WHAT TO EXPLAIN ALOUD

Explain that each case is assessed through the same three criteria: economics, feasibility, and online computation time. Keep this slide brief so most of the presentation time remains available for methods and results.

Why setpoint determination matters

MAIN MESSAGE

Storage and fluctuating inputs make today's decision affect tomorrow's flexibility.

WHAT TO PUT ON THE SLIDE

- Electricity prices change over time; the factory also receives fluctuating wind power.
- Charging now can support later demand, but it changes current cost and future storage availability.
- Decisions must satisfy power, storage, temperature, and process-coupling limits.
- Repeated nonlinear optimization motivates studying faster setpoint generation.

SUGGESTED VISUAL / LAYOUT

Show a price curve above a storage timeline. Use arrows to connect a low-price charging decision with later discharge. Place cost, constraints, and response time beneath the illustration.

WHAT TO EXPLAIN ALOUD

Define a setpoint with a concrete example, such as battery charging power or a thermal-storage bypass setting. Explain why a controller cannot optimize each hour independently. Frame computational cost as a motivation to investigate RL rather than proof that optimization is unusable.

Research question and success criteria

MAIN MESSAGE

Can RL produce feasible and cost-effective setpoints across different system complexities?

WHAT TO PUT ON THE SLIDE

- Display the central research question from the thesis prominently.
- Formulation: which states, continuous actions, and rewards represent the engineering task?
- Performance: how do SAC and TD3 compare with the optimization baseline?
- Applicability: where does fast inference help, and which feasibility limitations remain?

SUGGESTED VISUAL / LAYOUT

Place the research question in a large central box. Under it, show three equally sized criteria: economic objective, constraint satisfaction, and online time.

WHAT TO EXPLAIN ALOUD

Define success as a balance of all three criteria. A high reward alone does not establish good engineering performance because reward includes artificial penalties. Return to this question explicitly on the final slide.

SOURCE / paper/main.tex: Section 1, research question and subquestions

Contributions of the thesis

MAIN MESSAGE

The work provides a shared comparison framework for two continuous-control energy systems.

WHAT TO PUT ON THE SLIDE

- Formulate battery and cookie-factory scheduling as finite-horizon MDPs.
- Implement system-specific physical action mappings and train SAC and TD3 policies.
- Reduce the factory policy to a storage-mode command and mass flow; derive other setpoints through process relations.
- Compare original operating cost, separate constraint metrics, online runtime, and offline training effort.

SUGGESTED VISUAL / LAYOUT

Use four numbered contribution cards. Link the action-mapping card to a small schematic showing normalized policy output becoming physical setpoints.

WHAT TO EXPLAIN ALOUD

Describe the contribution as applying and evaluating established RL algorithms for these systems. Focus on the engineering formulation and comparison, and distinguish the contributions already demonstrated from the future improvements proposed later.

SOURCE / paper/main.tex: Sections 3, 4, and 5.1

Two routes to operating setpoints

MAIN MESSAGE

Optimization solves a horizon; a trained policy generates actions from observations.

WHAT TO PUT ON THE SLIDE

- Optimization baseline: model equations, objective, and hard constraints are supplied to IPOPT.
- RL training: repeated simulator interaction improves an actor-critic policy offline.
- RL operation: observation -> policy -> physical action map -> setpoint.
- Each policy is compared with IPOPT on identical inputs and the original economic objective.

SUGGESTED VISUAL / LAYOUT

Draw two parallel pipelines that end at the same system simulator. Shade the RL training loop separately from the online action-generation path. Label IPOPT as the thesis baseline.

WHAT TO EXPLAIN ALOUD

RL shifts computation into offline learning and still relies on the simulator. IPOPT is a local nonlinear solver, without a certified global optimum. Battery SAC and TD3 reports use separately sampled windows; all three factory methods share one window set.

SOURCE / paper/main.tex: Sections 2.2, 3.2.1, and 4.2.1

Experimental design across two cases

MAIN MESSAGE

Use the same evaluation logic while increasing physical and constraint complexity.

WHAT TO PUT ON THE SLIDE

- Battery: one continuous control command; price input; SOC dynamics.
- Factory: five independent optimization variables, reduced to two policy actions; price and wind inputs.
- One episode is 24 hourly decisions; both methods receive the same fixed horizon profiles.
- Battery: 1000 policy/IPOPT comparisons per algorithm, sampled separately. Factory: 200 shared windows for all three methods.

SUGGESTED VISUAL / LAYOUT

Create a two-column case-study comparison table with rows for storage, external inputs, action dimensions, horizon, and evaluation sample count.

WHAT TO EXPLAIN ALOUD

The factory optimizer and policy use different decision parameterizations. Full-horizon profiles remain fixed. The battery reports establish paired policy/IPOPT comparisons within each run, but do not establish a common SAC/TD3/IPOPT sample.

SOURCE / paper/main.tex: Sections 3.3 and 4.3; src/battery_model/model_comparison_SAC_final.txt and model_comparison_TD3_final.txt

From scheduling problem to MDP

MAIN MESSAGE

The observation must describe current flexibility and the remaining horizon.

WHAT TO PUT ON THE SLIDE

- State/observation: storage state, exogenous inputs, time position, and available forecasts.
- Action: continuous policy output mapped into physical operating decisions.
- Transition: the simulator updates storage and evaluates process equations.
- Reward: negative electricity cost with penalties for requirements not enforced directly.

SUGGESTED VISUAL / LAYOUT

Use the current paper/figures/markov_diagram_v2.pdf. Preserve the state x_t, action a_t, reward r_t, and next-state notation. Add only brief engineering labels around the interaction diagram.

WHAT TO EXPLAIN ALOUD

Walk through one hour of interaction. Explain that the terminal condition makes the remaining horizon relevant to the decision. Emphasize that learned actions are processed by engineering mappings before being applied to the simulated plant.

SOURCE / paper/main.tex: Sections 2.3, 3.2.1, and 4.2.1; Figure 1

Why compare SAC and TD3?

MAIN MESSAGE

Both learn continuous actions, with different exploration and actor-update mechanisms.

WHAT TO PUT ON THE SLIDE

- Shared structure: off-policy actor-critic learning, replay buffer, and twin critics.
- TD3: deterministic actor, delayed policy updates, and target-policy smoothing.
- SAC: stochastic actor with an entropy objective during training.
- Evaluation: TD3 uses its deterministic output; SAC uses the policy's deterministic mean action.

SUGGESTED VISUAL / LAYOUT

Use a compact SAC-versus-TD3 table. Add one shared actor -> action and critic -> value diagram instead of reproducing full update equations.

WHAT TO EXPLAIN ALOUD

Explain the actor as the setpoint generator and the critic as an estimate of long-term value. Avoid assuming one algorithm must win because of its design. The results determine the comparison, and the thesis does not establish a universal algorithm ranking.

SOURCE / paper/main.tex: Sections 2.3.1 and 3.2.3

Offline training setup and checkpoint settings

MAIN MESSAGE

Train on random 24-hour windows; distinguish reported settings from saved checkpoint metadata.

WHAT TO PUT ON THE SLIDE

- Gymnasium environments and Stable-Baselines3; one environment instance per training run.
- 50 tuning trials; replay buffer 200000; mini-batch 256; target update tau 0.005.
- Linear learning rate starts at 0.0003; 24-step episodes; preliminary SAC schedule study used 32 runs each.
- Saved cookie checkpoints use gamma=1; battery candidates use gamma=0.99 or 0.9999. Battery benchmark checkpoint identity is unrecorded.

SUGGESTED VISUAL / LAYOUT

Use a compact table of framework, replay, batch, learning rate, tau, and horizon. Add a separate checkpoint note for discount and network settings; avoid a universal architecture claim.

WHAT TO EXPLAIN ALOUD

Saved factory architectures are SAC 256-256 and TD3 400-300, with gamma=1. Battery candidates vary in architecture and gamma; the benchmark checkpoint is unrecorded. Reconcile artifacts and manuscript settings before asserting a shared configuration. Learning-rate differences were small relative to run variation.

SOURCE / paper/main.tex: Sections 3.2.2 and 3.3, Table 1; saved checkpoint metadata

What is measured?

MAIN MESSAGE

Report physical economics, feasibility, and runtime separately from training reward.

WHAT TO PUT ON THE SLIDE

- Grid objective: sum(price x grid power x time step / 1000), with EUR/MWh prices and kW power.
- Lower objective means higher battery profit or lower factory operating cost.
- Report each constraint residual or violation in its own physical unit.
- Compare complete 24-hour RL rollout time with optimization solve time; list training effort separately.

SUGGESTED VISUAL / LAYOUT

Show three metric cards with units: EUR, constraint-specific units, and seconds. Add a small note: economic objective excludes reward penalties. Define the signed gap as J_RL minus J_math.

WHAT TO EXPLAIN ALOUD

Explain the battery sign convention before showing negative cost values. Do not combine temperature residuals and power violations into a physical total with mixed units. The factory absolute relative gap is averaged over windows, so it is not the percentage difference of the aggregate mean costs.

SOURCE / paper/main.tex: Sections 3.2.3 and 4.2.2; Tables 2-3

Battery system and operating task

MAIN MESSAGE

The battery buys low-price energy and sells at higher prices while managing SOC.

WHAT TO PUT ON THE SLIDE

- Use the grid-connected battery schematic and define SOC as stored-energy fraction.
- Positive grid power means charging; negative grid power means discharge and sale.
- The displayed mapping uses maximum power 20 kW and efficiency 0.9.
- The end-of-horizon requirement is SOC_final >= 0.50; the experiments use no buy-sell price spread.

SUGGESTED VISUAL / LAYOUT

Reuse paper/figures/battery_model.pdf. Label charge and discharge arrows, electricity price, SOC, and the terminal target. Place the operating objective next to the diagram.

WHAT TO EXPLAIN ALOUD

Describe how charging losses and storage limits alter simple price arbitrage. Explain that the terminal requirement prevents the controller from benefiting merely by leaving the battery depleted at the end of the scheduling horizon.

SOURCE / paper/main.tex: Section 3.1; Figure 2

Battery MDP: observation and action

MAIN MESSAGE

One normalized command controls charging or discharge using the full horizon price profile.

WHAT TO PUT ON THE SLIDE

- Observation: current SOC, normalized current price, normalized horizon position, and the fixed 24-step price profile.
- Action: scalar u in [-1, 1]; positive for charging, negative for discharging.
- Transition: SOC_next = SOC + storage-side power x time step / energy capacity.
- After efficiency conversion, limit storage-side power to available energy and remaining capacity before the SOC update.

SUGGESTED VISUAL / LAYOUT

Draw observation -> policy -> smooth power/ efficiency map -> storage-power limit -> SOC update. Display the SOC equation beneath the flow; keep each stage legible.

WHAT TO EXPLAIN ALOUD

Commanded action, grid power, and storage power differ. If the storage-power limit changes the request, recompute grid power through the inverse efficiency map before calculating cost. The full price profile remains fixed throughout the horizon.

Battery feasibility mapping and reward

MAIN MESSAGE

Storage-power limits enforce intermediate SOC bounds; penalties encourage terminal SOC.

WHAT TO PUT ON THE SLIDE

- Smooth action/SOC gates suppress discharge near empty storage and charge near full storage.
- Clip storage-side power to [-SOC x E_max / dt, (1-SOC) x E_max / dt]; then recompute grid power if needed.
- Reward is negative electricity cost minus a final SOC-shortfall penalty.
- A second variant adds a progressive penalty when SOC becomes insufficient to retain terminal reachability.

SUGGESTED VISUAL / LAYOUT

Refresh paper/figures/battery_smooth_mappings.pdf from the current file. Add a storage-power-limit callout and compact reward text. Keep the progressive-reachability variant separate; Figure 4 can support its explanation.

WHAT TO EXPLAIN ALOUD

SOC gates attenuate requests but do not alone guarantee next-step feasibility. The storage-power limit preserves intermediate SOC bounds, while terminal SOC remains a soft requirement. Mapping parameters are k=700, deadband=0.0025, and efficiency scale=1 kW.

SOURCE / paper/main.tex: Sections 3.1 and 3.2.1; Figures 3-4; src/battery_model/battery_env.py

Battery economics: separate paired comparisons

MAIN MESSAGE

Both policies approach IPOPT in separate 1000-window comparisons; TD3 has the smaller reported paired gap.

WHAT TO PUT ON THE SLIDE

- TD3 run: baseline mean profit 3.1851 EUR; TD3 mean profit 3.0223 EUR.
- TD3 paired objective gap: +0.1628 +/- 0.1284 EUR over 1000 windows.
- SAC run: baseline mean profit 3.1126 EUR; SAC mean profit 2.8911 EUR.
- SAC paired objective gap: +0.2215 +/- 0.0811 EUR over a separately sampled 1000-window run.

SUGGESTED VISUAL / LAYOUT

Use two paired rows or panels, one per algorithm, each with its own baseline. Label profit=-J and gaps as mean +/- SD. Do not use a common three-bar baseline chart.

WHAT TO EXPLAIN ALOUD

The reports sample windows separately, so they support within-run policy/IPOPT gaps. A direct common-window SAC/TD3 ranking needs a shared sample. The manuscript's SAC gap combines means from different runs; use the paired report value here.

SOURCE / src/battery_model/model_comparison_TD3_final.txt and model_comparison_SAC_final.txt; paper/main.tex: Section 3.3, Table 2

Battery dispatch and the rule-based example

MAIN MESSAGE

Learned scheduling uses storage flexibility more effectively than the illustrated fixed thresholds.

WHAT TO PUT ON THE SLIDE

- Show the TD3-versus-optimization battery power and SOC trajectories for one 24-hour window.
- Explain low-price charging, higher-price discharge, and the final storage state.
- Fixed thresholds: charge at <= 10 EUR/MWh and discharge at >= 25 EUR/MWh, with a terminal-SOC safeguard.
- On this matched window: TD3 profit 2.33 EUR; baseline 2.73 EUR; threshold 1.93 EUR. TD3 improves on thresholds by about 20.4%.

SUGGESTED VISUAL / LAYOUT

Replace both embedded plots with current battery_model_dispatch_plot_td3.png and battery_fixed_threshold_dispatch.png. Arrange them side by side at equal size, with matched price-window and profit labels beneath each.

WHAT TO EXPLAIN ALOUD

The terminal safeguard can override fixed thresholds to restore SOC to 0.50, explaining late charging above the purchase threshold. The 20.4% improvement uses unrounded results on one matched window; it is not a 1000-window claim.

SOURCE / paper/main.tex: Section 3.3; Figures 6-7

Battery terminal feasibility versus economics

MAIN MESSAGE

The progressive penalty removes observed terminal violations at a larger objective gap.

WHAT TO PUT ON THE SLIDE

- TD3 with terminal penalty only: 40/1000 windows violated terminal SOC, an observed rate of 4%.
- Mean terminal-SOC shortfall across all 1000 windows was 0.000252, equivalent to 0.0504 kWh.
- With the progressive penalty: observed violation rate 0%; mean objective gap 0.2436 +/- 0.1473 EUR.
- With terminal penalty only: mean objective gap 0.1628 +/- 0.1284 EUR; gaps are mean +/- standard deviation.

SUGGESTED VISUAL / LAYOUT

Use a TD3-only comparison table: terminal penalty versus terminal plus progressive penalty. Include 40/1000 for the terminal-only report; do not assign that count to the alternative experiment.

WHAT TO EXPLAIN ALOUD

The thesis selects terminal penalty only for its higher profit, accepting small shortfalls. The shortfall mean includes all windows; objective gaps are mean +/- SD. Zero observed violations with progressive feedback is empirical, and its sample count is unrecorded.

SOURCE / paper/main.tex: Section 3.3; src/battery_model/model_comparison_TD3_final.txt

Battery runtime and training effort

MAIN MESSAGE

TD3 produces a complete rollout in about 8 milliseconds after offline training.

WHAT TO PUT ON THE SLIDE

- Mean complete-rollout time: TD3 0.0082 s; SAC 0.0126 s.
- Use the single manuscript IPOPT mean runtime: 0.5605 s, alongside TD3 0.0082 s and SAC 0.0126 s.
- The manuscript reports TD3 online speedup of approximately 68.13 times.
- Active offline training: TD3 23 minutes; SAC 36 minutes, excluded from online runtime.

SUGGESTED VISUAL / LAYOUT

Use a two-column method/runtime table with IPOPT, TD3, and SAC, displaying IPOPT once as in manuscript Table 2. Show offline training separately and label timings as complete 24-hour schedules.

WHAT TO EXPLAIN ALOUD

Measurements use the thesis hardware, AMD Ryzen 7 7840HS. Present the mean runtimes from manuscript Table 2. Model loading and training are excluded; displayed times are rounded, while the reported TD3 speedup uses original measurements.

SOURCE / paper/main.tex: Sections 3.2.2 and 3.3; Table 2

Cookie-factory energy utility system

MAIN MESSAGE

Wind, a high-temperature heat pump, and thermal storage supply process steam.

WHAT TO PUT ON THE SLIDE

- Introduce the wind turbine, high-temperature heat pump, thermal energy storage, and steam generator.
- Wind reduces the grid electricity required to operate the heat-pump system.
- Controllable bypasses route heat-transfer fluid through storage or toward the steam generator.
- The operating goal is to meet steam demand at minimum grid-electricity cost.

SUGGESTED VISUAL / LAYOUT

Use current paper/figures/cookie_steam_system.pdf. Distinguish electrical and thermal paths and preserve attribution: adapted from Kyriakidis et al. (2024), Figure 2; system originally introduced by Walden et al. (2023).

WHAT TO EXPLAIN ALOUD

Explain the energy utility that serves production, rather than the cookie manufacturing process itself. Identify TES as the source of time-coupled flexibility. Introduce component abbreviations once before using them in the next method slides.

SOURCE / paper/main.tex: Section 4.1; Figures 8-9 and their attribution

Why the factory problem is harder

MAIN MESSAGE

Nonlinear components and coupled storage/process requirements create a restricted feasible region.

WHAT TO PUT ON THE SLIDE

- The optimizer independently selects beta1, beta2, compressor-speed variable R, mass flow, and inlet temperature.
- Polynomial surrogate models describe heat-pump demand and heat-exchanger temperatures.
- Constraints couple power balance, bypass mixing, storage heat flow, and storage temperature.
- Terminal TES temperature must return to its initial value; charging and discharging are mutually exclusive.

SUGGESTED VISUAL / LAYOUT

Use current paper/figures/cookie_detailed_flow_chart.pdf as a readable flow inset, with the thesis notation. Attribute the adaptation to Kyriakidis et al. (2024), Figure 3, and original system to Walden et al. (2023).

WHAT TO EXPLAIN ALOUD

Explain why independent bounds alone cannot ensure feasibility: a bounded flow and speed can still violate a temperature coupling or grid-power requirement. Keep the complete equation system in the thesis; show only the storage balance or terminal equality if needed.

SOURCE / paper/main.tex: Section 4.1; Figure 9

Factory MDP and reduced action space

MAIN MESSAGE

The policy selects two quantities while the environment derives the physical operating point.

WHAT TO PUT ON THE SLIDE

- Observation includes TES temperature, price, wind, time position, terminal gap, previous controls and diagnostics, and price/wind forecasts.
- Policy actions: signed storage-mode command b in [-1, 1] and mass flow in [5, 16].
- b < -0.05 selects discharge; |b| <= 0.05 selects idle; b > 0.05 selects charge.
- The inactive storage path is bypassed, so simultaneous charging and discharging is prevented by the mapping.

SUGGESTED VISUAL / LAYOUT

Draw a three-region horizontal mode scale and a separate mass-flow slider. Show the two policy outputs expanding into beta1, beta2, inlet temperature, and derived R through environment equations.

WHAT TO EXPLAIN ALOUD

Highlight the reduction from five independent optimization variables to two RL outputs. This encodes some engineering structure but restricts how the policy explores operating points. The 0.05 idle half-width was fixed empirically and was not separately optimized.

SOURCE / paper/main.tex: Sections 4.1 and 4.2.1; storage-mode mapping

Deriving compressor speed from process coupling

MAIN MESSAGE

Solving the outlet-temperature relation reduces action dimension but can leave residual error.

WHAT TO PUT ON THE SLIDE

- Use bypass settings, mass flow, storage state, and mixing equations to derive inlet temperature and the required outlet temperature.
- Solve the cubic heat-exchanger surrogate relation for R within [0.8, 1.53].
- If multiple bounded real roots exist, choose the one closest to the previously applied R; initial reference is 1.165.
- If no bounded root exists, apply the bounded value with the smallest mismatch and record the F4 residual.

SUGGESTED VISUAL / LAYOUT

Show a decision flow: derive required temperature -> solve cubic -> bounded root? -> nearest-root selection or minimum-residual fallback. Label the fallback as potentially infeasible.

WHAT TO EXPLAIN ALOUD

Explain that deriving R is stronger than letting the actor guess an independent speed, but it does not guarantee a feasible root for every mode/flow command. The recorded residual must remain visible in evaluation even when a bounded fallback is applied.

SOURCE / paper/main.tex: Section 4.2.1; required outlet temperature and R derivation

Factory reward and constraint handling

MAIN MESSAGE

Normalized penalties discourage constraints that the action map does not guarantee.

WHAT TO PUT ON THE SLIDE

- Reward = negative grid cost minus normalized state, inlet-temperature, heat-flow, F4, grid-power, and terminal-cycle penalties.
- Training bounds: TES 183-324 degrees C; inlet temperature 177-250 degrees C.
- Maximum heat-flow and grid-power bounds are each 5000 kW.
- Weights: state 20; inlet 10; heat flow 5000; F4 500; grid power 3000; terminal cycle 3000.

SUGGESTED VISUAL / LAYOUT

Use two boxes: requirements handled structurally and requirements penalized in reward. Place the normalized reward components below; avoid a long equation with every parameter on the main slide.

WHAT TO EXPLAIN ALOUD

Explain why normalization allows weights to express relative priorities. These are empirical engineering choices rather than proven optimal coefficients. Large penalty weights still do not guarantee feasibility; show the measured remaining violations on slide 26.

SOURCE / paper/main.tex: Section 4.2.1; cookie reward and penalty coefficients

Factory economics over 200 matched windows

MAIN MESSAGE

TD3 has the smaller RL cost gap, while optimization retains the best mean objective.

WHAT TO PUT ON THE SLIDE

- Mean cost: Optimization baseline 2012.2433 EUR; TD3 2081.3496 EUR; SAC 2092.5760 EUR.
- Mean signed cost gap: TD3 69.1063 EUR; SAC 80.3327 EUR.
- Mean absolute relative gap: TD3 5.66%; SAC 6.31%.
- All three methods use identical price and wind profiles for the 200 comparison windows.

SUGGESTED VISUAL / LAYOUT

Use a readable three-column cost table or mean-cost chart, with separate gap and variability callouts. Add a feasibility qualification beside the economics; omit a small ten-window thumbnail.

WHAT TO EXPLAIN ALOUD

Describe these as objective differences. The RL schedules contain violations, so their gaps are not optimality gaps between equally feasible solutions. The mean absolute relative gap is a per-window statistic and differs from the percentage gap between mean costs.

SOURCE / paper/main.tex: Section 4.3; Table 3 and Figure 10

Factory dispatch: where the cost gap arises

MAIN MESSAGE

The example shows imperfect load shifting and a negative-grid-power violation in TD3's schedule.

WHAT TO PUT ON THE SLIDE

- Show wind power, heat-pump demand, grid power, and electricity price for one 24-hour window.
- Compare TD3 and optimization-baseline grid-power schedules directly.
- Example cost: TD3 1244.27 EUR; optimization 1130.70 EUR; difference 113.57 EUR, about 10.0%.
- Mark the high-price purchases and TD3's negative grid power around hours 12-15, violating the nonnegative grid-power bound.

SUGGESTED VISUAL / LAYOUT

Use current paper/figures/cookie_factory_dispatch_plot_td3.png. Align time axes, add the cost callout, and highlight the hours 12-15 segment below zero in the grid-power panel.

WHAT TO EXPLAIN ALOUD

The baseline shifts purchases toward cheaper hours more effectively. TD3 also produces inadmissible negative grid power when wind exceeds modeled demand. The cost difference therefore compares schedules with different feasibility; this is one illustrative window.

SOURCE / paper/main.tex: Section 4.3; Figure 11 and accompanying example

Factory feasibility: the main remaining limitation

MAIN MESSAGE

Both policies violate process requirements; the optimization baseline reports zero violations.

WHAT TO PUT ON THE SLIDE

- Mean per-window F4 residual sum [degrees C]: SAC 0.4555; TD3 0.5300; baseline 0.
- Mean per-window heat-flow-bound violation sum [kW]: SAC 0.5989; TD3 3.5719; baseline 0.
- Mean per-window grid-power violation sum [kW]: SAC 98.3749; TD3 68.5787; baseline 0.
- Mean absolute terminal endpoint error [degrees C]: SAC 1.8380; TD3 2.1386; baseline 0. State/inlet-bound violations average zero.

SUGGESTED VISUAL / LAYOUT

Use a table with category and unit per row. Label operating metrics as mean per-window sums and the terminal metric as mean absolute endpoint error. Highlight the smaller policy value in each row.

WHAT TO EXPLAIN ALOUD

These are averages over all windows, not averages conditional on a violation. TD3 has smaller grid-power violations; SAC has smaller coupling, heat-flow, and terminal errors. Different physical units cannot be combined into a total violation; economic preference does not imply uniform feasibility superiority.

SOURCE / paper/main.tex: Section 4.3; Table 3 and feasibility discussion

Factory runtime and offline cost

MAIN MESSAGE

Factory RL rollouts are much faster online, with feasibility still unresolved.

WHAT TO PUT ON THE SLIDE

- Mean full-schedule runtime: optimization 11.5901 s; TD3 0.0180 s; SAC 0.0217 s.
- Reported online speedups: TD3 approximately 645.06 times; SAC approximately 533.81 times.
- Active offline training: TD3 1.24 hours; SAC 2.81 hours.
- RL timing includes reset, prediction, transitions with root solving, and stepwise residual checks; loading and training are excluded.

SUGGESTED VISUAL / LAYOUT

Show online runtimes on a logarithmic chart. Display training hours separately and add a small feasibility reminder beside the speedup callout. Retain the hardware and timing-scope caption.

WHAT TO EXPLAIN ALOUD

TD3's approximately 18 ms includes the complete rollout and environment calculations. Future feasibility restoration would add work. Speedups are ratios of mean runtimes on AMD Ryzen 7 7840HS; tabulated times are rounded. Training and model loading are excluded.

SOURCE / paper/main.tex: Sections 3.2.2 and 4.3; Table 3 and timing scope

What changes as complexity increases?

MAIN MESSAGE

Fast setpoint generation persists, while economic and feasibility differences become more consequential.

WHAT TO PUT ON THE SLIDE

- Battery TD3: mean objective gap 0.1628 EUR and reported online speedup 68.13 times; small terminal shortfalls remain in the selected variant.
- Factory TD3: mean cost gap 69.1063 EUR, mean absolute relative gap 5.66%, and reported speedup 645.06 times; several constraint categories remain nonzero.
- The study uses simulators and known, fixed 24-hour profiles, without real-plant validation.
- Battery SAC/TD3 samples are separate; an independent held-out test and forecast-error robustness remain unestablished.

SUGGESTED VISUAL / LAYOUT

Use a two-case table with economics, feasibility, and speed. Add brief scope notes for fixed forecasts, simulator dependence, separate battery samples, and evaluation-data independence.

WHAT TO EXPLAIN ALOUD

Raw EUR gaps have different economic scales. Models, action parameterizations, and constraints also differ, so the results do not isolate complexity as the cause. The battery paired gaps come from separate samples; an unseen-data or forecast-error claim requires further evaluation.

SOURCE / paper/main.tex: Sections 3.3, 4.3, and 5; battery comparison reports

Next steps toward reliable operation

MAIN MESSAGE

Combine rapid learned suggestions with stronger feasibility handling and realistic validation.

WHAT TO PUT ON THE SLIDE

- Prioritize constraint-preserving mappings or action projection for factory grid-power and process violations.
- Investigate the reduced action parameterization and derived-R root selection further.
- Test rolling-horizon control under forecast errors, noise, disturbances, and model-plant mismatch.
- Evaluate RL warm starts for IPOPT, BO-IPOPT comparisons, and higher-fidelity or real-plant validation.

SUGGESTED VISUAL / LAYOUT

Show a proposed workflow: RL suggestion -> feasibility restoration / optimization refinement -> applied setpoint. Add a short staged validation path from simulator to realistic disturbances to plant tests.

WHAT TO EXPLAIN ALOUD

Make clear that this hybrid workflow is proposed future work and was not evaluated in the reported results. Tie each next step to an observed limitation: constraint violations, fixed forecasts, or simulator dependence.

SOURCE / paper/main.tex: Section 5.2, Future Work

Conclusion and questions

MAIN MESSAGE

RL learns economically meaningful, very fast setpoints; reliable feasibility remains essential.

WHAT TO PUT ON THE SLIDE

- Answer the research question: RL can provide useful continuous setpoints, with a qualified feasibility result.
- Battery TD3 approaches IPOPT in its paired run; factory TD3 has the smaller RL economic gap and faster rollout on shared windows.
- IPOPT retains better mean economics and feasibility. Battery algorithm comparisons use separate sampled runs.
- Close with the priority: stronger constraint handling and realistic validation. Add 'Thank you - Questions?'.

SUGGESTED VISUAL / LAYOUT

Use three short takeaway lines and a small battery/factory summary. Keep the final conclusion qualified by feasibility and sample scope; cite Section 5.1 and retain Ashish Thapa's name in the footer.

WHAT TO EXPLAIN ALOUD

Return to economics, feasibility, and time. Explain the battery terminal-shortfall choice, separate battery samples, checkpoint provenance, and factory action reduction if asked. Low cost from an infeasible schedule is insufficient; the result supports fast learned suggestions with further validation.

SOURCE / paper/main.tex: Abstract and Section 5.1; battery comparison reports

What makes the scheduling task an MDP?

MAIN MESSAGE

A Markov state contains the information needed to predict the next state after an action.

WHAT TO PUT ON THE SLIDE

- MDP ingredients: state space E, action space A, admissible actions D_t(x), transition Q_t, reward r_t, and terminal reward g_N.
- Markov property: P(x_next | history, a_t) = P(x_next | x_t, a_t).
- A policy pi_t(a | x) selects an action distribution; a deterministic policy selects one action mu_t(x).
- Battery example: SOC, current price, horizon position, and the fixed full price profile describe the remaining scheduling task.

SUGGESTED VISUAL / LAYOUT

Use a compact table of MDP ingredients and meanings, retaining Q_t for the transition kernel. Place the Markov-property equation and a battery-state example beneath it.

WHAT TO EXPLAIN ALOUD

Purpose: define the formal decision problem. Optional placement: before slide 8. Time and forecast information belong in the state because decisions depend on the remaining horizon. An incomplete observation need not satisfy the Markov property.

SOURCE / paper/main.tex: Section 2.3, Markov decision model; Figure 1

Finite and infinite planning horizons

MAIN MESSAGE

The 24-hour endpoint changes the decision problem and the meaning of future value.

WHAT TO PUT ON THE SLIDE

- Finite horizon: t=0,...,N-1, with a terminal state x_N and boundary reward g_N(x_N).
- Infinite horizon: decisions continue indefinitely; bounded rewards with gamma<1 have a finite discounted sum. Average-reward formulations are another option.
- Here N=24 represents hourly scheduling over one input window. Battery terminal SOC and the factory storage cycle prevent gains from leaving storage depleted.
- A finite horizon allows gamma=1 mathematically; this does not establish the discount used in every saved model.

SUGGESTED VISUAL / LAYOUT

Compare a 24-step timeline ending at a terminal requirement with an open-ended timeline. Mark the terminal SOC or TES-cycle condition at the finite endpoint.

WHAT TO EXPLAIN ALOUD

Purpose: explain the scheduling boundary. Optional placement: before slide 8 or 10. The horizon is the experimental scheduling choice. Gamma=1 is valid for a finite return, but checkpoint metadata and manuscript settings must still be reconciled.

SOURCE / paper/main.tex: Section 2.3, finite-horizon return and terminal reward

Return, state value, and action value

MAIN MESSAGE

Value functions estimate the future reward associated with a state or a chosen action.

WHAT TO PUT ON THE SLIDE

- Return: G_t = sum(i=t,...,N-1) gamma^(i-t) r_i + gamma^(N-t) g_N(x_N).
- State value: v_t^pi(x) = E_pi[G_t | x_t=x], before the action is selected.
- Action value: q_t^pi(x,a) = E_pi[G_t | x_t=x, a_t=a], after choosing a particular action.
- In these environments g_N=0: the final transition reward already contains the terminal penalty. Return combines economics and penalties.

SUGGESTED VISUAL / LAYOUT

Use three equation cards for G, v, and q. Add a small battery choice example: identical SOC, different charge/discharge decisions, different future value.

WHAT TO EXPLAIN ALOUD

Purpose: distinguish a realized return from its expected values. Optional placement: before slide 9. The critic estimates q for continuous setpoints. Economic objective J remains separate because the reward also includes feasibility penalties.

SOURCE / paper/main.tex: Section 2.3, return and value-function definitions

Bellman expectation and optimality equations

MAIN MESSAGE

Bellman recursion separates immediate reward from the value of subsequent decisions.

WHAT TO PUT ON THE SLIDE

- For a fixed policy: q_t^pi(x,a) = r_t(x,a) + gamma E[v_(t+1)^pi(x_next) | x,a].
- Policy evaluation: v_t^pi(x) = E_(a~pi_t)[q_t^pi(x,a)]. A deterministic policy evaluates its selected action.
- Optimality: q_t*(x,a) = r_t(x,a) + gamma E[v_(t+1)*(x_next) | x,a]; v_t*(x) = sup_(a in D_t(x)) q_t*(x,a).
- Boundary: v_N(x)=g_N(x). Expectation equations evaluate a specified policy; optimality equations select the best attainable action.

SUGGESTED VISUAL / LAYOUT

Use two parallel panels: policy-weighted averaging and best-action selection. Place the common immediate-reward plus future-value recursion across the top.

WHAT TO EXPLAIN ALOUD

Purpose: explain the recursion behind value learning. Optional placement: before slide 9. Expectations account for transitions and policy randomness. These ideal equations do not prove that neural-network RL finds the optimum or preserves engineering feasibility.

SOURCE / paper/main.tex: Section 2.3, Bellman equations and RL solution approaches

Temporal-difference learning and the critic

MAIN MESSAGE

The critic learns from observed reward and a bootstrapped estimate of the remaining return.

WHAT TO PUT ON THE SLIDE

- Replay transition: (x_t,a_t,r_t,x_next,d_t), with d_t=1 for a terminal transition.
- One-step TD target: y_t = r_t + gamma (1-d_t) v_target(x_next).
- TD error: delta_t = y_t - q_phi(x_t,a_t). Train the critic by minimizing E[delta_t^2], treating the target as fixed.
- At the true 24-hour endpoint d_t=1, so y_t=r_t. The returned final reward already includes the terminal penalty.

SUGGESTED VISUAL / LAYOUT

Draw one transition feeding two paths: observed reward and target-network continuation value. Show their sum, the predicted q, and the resulting TD error.

WHAT TO EXPLAIN ALOUD

Purpose: explain how sampled experience trains the critic. Optional placement: before slide 9 or 10. Bootstrapping uses an estimate instead of waiting for a full return. Here the finite endpoint is terminal; an external time truncation requires separate treatment.

Policy gradients and actor-critic learning

MAIN MESSAGE

The actor improves continuous decisions using the critic's estimate of long-term value.

WHAT TO PUT ON THE SLIDE

- Stochastic policy-gradient principle: grad_theta J approximately E[grad_theta log pi_theta(a|x) A^pi(x,a)].
- Advantage: A^pi(x,a)=q^pi(x,a)-v^pi(x). A state baseline can reduce gradient variance without changing its expectation.
- Deterministic policy gradient: grad_theta J approximately E[grad_a q_phi(x,a)|_(a=mu_theta(x)) grad_theta mu_theta(x)].
- Actor-critic: critic fits TD targets; actor follows a value-based gradient. SAC instead uses a reparameterized entropy-regularized actor objective.

SUGGESTED VISUAL / LAYOUT

Use a paired actor/critic diagram. Show the stochastic score-gradient principle and deterministic chain rule in separate cards; label SAC's objective as an algorithm-specific extension.

WHAT TO EXPLAIN ALOUD

Purpose: connect value learning to action selection. Optional placement: before slide 9. The generic score-gradient equation is not SAC's implementation update. SAC differentiates through reparameterized samples of its policy while optimizing q minus alpha log pi.

SOURCE / paper/main.tex: Sections 2.3 and 2.3.1, policy gradients and SAC actor objective

On-policy, off-policy, and replay

MAIN MESSAGE

Off-policy learning can reuse transitions generated by earlier or exploratory policies.

WHAT TO PUT ON THE SLIDE

- Behavior policy pi_b collects data; the current target policy is the policy being improved.
- On-policy updates use current-policy data. Off-policy updates allow behavior and target policies to differ.
- SAC and TD3 store transitions in replay buffer D and train repeatedly on sampled minibatches, improving data reuse and reducing temporal correlation.
- Target networks change slowly: target <- (1-tau) target + tau learned. Replay actions and actions used in learning targets can come from different policies.

SUGGESTED VISUAL / LAYOUT

Draw collection -> replay buffer -> critic and actor updates. Use distinct labels for the stored behavior action and the current/target actor's next action.

WHAT TO EXPLAIN ALOUD

Purpose: explain the shared learning structure. Optional placement: before slide 9 or 10. Off-policy does not mean ignoring the current policy: SAC samples from it, while TD3 uses a target actor for the critic target. Target networks help stabilize bootstrapping.

SOURCE / paper/main.tex: Sections 2.3 and 2.3.1, off-policy learning and target updates

Double Q-learning and clipped twin critics

MAIN MESSAGE

Separating estimates reduces the effect of selecting actions from noisy value predictions.

WHAT TO PUT ON THE SLIDE

- Maximizing a noisy value estimate tends to favor overestimated actions, creating maximization bias.
- Classical Double Q-learning uses one estimator to select an action and another to evaluate it; it does not simply take their minimum.
- TD3 selects the next action with a target actor and takes min(q1_target,q2_target) in the TD target. Its actor maximizes q1.
- SAC also uses twin critics and their minimum, with policy-sampled next actions and an entropy term. Conservative estimates can introduce underestimation.

SUGGESTED VISUAL / LAYOUT

Compare three compact selection/evaluation diagrams: a single noisy maximum, Double Q's split roles, and TD3/SAC's minimum of twin critic estimates.

WHAT TO EXPLAIN ALOUD

Purpose: distinguish related bias-reduction ideas. Optional placement: before slide 9. Classical Double Q-learning and clipped double Q-learning have different target constructions. Twin critics reduce overestimation pressure but do not certify accuracy or optimality.

SOURCE / paper/main.tex: Section 2.3.1; Double Q-learning and clipped double Q-learning

How the foundations become SAC and TD3 updates

MAIN MESSAGE

Both algorithms fit twin critics to TD targets, but use different next actions and actor objectives.

WHAT TO PUT ON THE SLIDE

- TD3: a_next=clip(mu_target(x_next)+clipped_noise,-1,1); y=r+gamma(1-d)min_j qj_target(x_next,a_next).
- TD3 actor maximizes E[q1(x,mu(x))]; actor and target updates are delayed relative to critic updates.
- SAC: a_next~pi_current; y=r+gamma(1-d)[min_j qj_target(x_next,a_next)-alpha log pi_current(a_next|x_next)].
- SAC actor maximizes E[min_j qj(x,a)-alpha log pi(a|x)] using policy samples. Target critics use (1-tau)old+tau learned.

SUGGESTED VISUAL / LAYOUT

Show explicit TD3 and SAC next-action and critic-target equations in two panels, followed by a compact actor-objective explanation. Add a shared legend for gamma, terminal mask d, alpha, and target networks; use q for action values.

WHAT TO EXPLAIN ALOUD

Purpose: provide an algorithm-detail option. Optional placement: after slide 9 and before slide 10. TD3 smoothing is target noise, distinct from collection noise; SAC uses reparameterized actor samples. Equations use normalized actions and omit future value at terminal transitions.

SOURCE / paper/main.tex: Section 2.3.1, TD3 and SAC update equations